

Backend Authentication & Authorization - Technical Changes Summary

Overview

We implemented a **JWT-based authentication system** with **BCrypt password hashing** and **role-based authorization** for the Article Platform. Here's a comprehensive breakdown of all technical changes:

1. Database Changes

Added `users` Table

```
CREATE TABLE users (  
    id BIGSERIAL PRIMARY KEY,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    phone VARCHAR(20) UNIQUE,  
    password_hash VARCHAR(255) NOT NULL,  
    role VARCHAR(255) NOT NULL CHECK (role IN ('ROLE_USER', 'ROLE_ADMIN')),  
    created_at TIMESTAMP,  
    updated_at TIMESTAMP  
);
```

Key Design Decisions:

- `password_hash` stores BCrypt hash (includes salt internally)
- `role` enum for authorization (future extensibility)
- Unique constraints on `username`, `email`, `phone`

Added Relationship to `articles` Table

```
ALTER TABLE articles ADD COLUMN author_id BIGINT REFERENCES users(id);
```

This establishes the **one-to-many** relationship: one user can have many articles.

Added References Table (Already Existed)

```
CREATE TABLE article_references (  
    article_id BIGINT REFERENCES articles(id),  
    reference_id BIGINT REFERENCES articles(id),  
    PRIMARY KEY (article_id, reference_id)  
);
```

2. New Dependencies

`pom.xml` Additions

Dependency	Purpose
<code>spring-boot-starter-security</code>	Spring Security framework
<code>jjwt-api</code> , <code>jjwt-impl</code> , <code>jjwt-jackson</code>	JWT generation & validation
<code>springdoc-openapi-starter-webmvc-ui</code>	Swagger/OpenAPI documentation

3. New Model/Entity

User.java Entity

```
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(unique = true, nullable = false)
    private String email;

    @Column(unique = true)
    private String phone;

    @Column(name = "password_hash", nullable = false)
    private String passwordHash; // BCrypt hash (salt embedded)

    @Enumerated(EnumType.STRING)
    private Role role; // ROLE_USER or ROLE_ADMIN

    @OneToMany(mappedBy = "author")
    private List<Article> articles;
}
```

Role.java Enum

```
public enum Role {
    ROLE_USER,
    ROLE_ADMIN;
}
```

4. Security Infrastructure

A. SecurityConfig.java - Core Security Configuration

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) {
        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/articles/**").permitAll()
                .requestMatchers(HttpMethod.POST, "/api/articles").authenticated()
                .requestMatchers("/api/users/**").authenticated()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
    }
}
```

Key Security Decisions:

- **CSRF disabled** – Stateless REST API doesn't need CSRF protection
- **Session STATELESS** – No session creation, each request is independent
- **JWT filter added** – Validates tokens before UsernamePasswordAuthenticationFilter

B. PasswordService.java - Password Hashing

```
@Service
public class PasswordService {
    private final BCryptPasswordEncoder passwordEncoder = new BCryptPasswordEncoder(12);

    public String hashPassword(String plainPassword) {
        return passwordEncoder.encode(plainPassword); // Auto-generates salt
    }

    public boolean verifyPassword(String plainPassword, String storedHash) {
        return passwordEncoder.matches(plainPassword, storedHash); // Extracts salt from hash
    }
}
```

Why BCrypt?

- Automatically generates random salt per password
- Salt is embedded in the hash (no separate column needed)
- Configurable work factor (12) makes brute-force expensive
- Industry standard for password storage

C. JwtService.java & JwtServiceImpl.java - JWT Management

```
@Service
public class JwtServiceImpl implements JwtService {

    @Value("${jwt.secret}")
    private String secretKey;

    @Value("${jwt.expiration}")
    private Long expirationMs; // 24 hours (86400000 ms)

    public String generateToken(User user) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("userId", user.getId());
        claims.put("role", user.getRole().name());

        return Jwts.builder()
            .claims(claims)
            .subject(user.getUsername())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expirationMs))
            .signWith(getSigningKey())
            .compact();
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        return extractUsername(token).equals(userDetails.getUsername()) && !isTokenExpired(token);
    }
}
```

JWT Structure:

```
Header: { "alg": "HS256", "typ": "JWT" }
Payload: { "sub": "username", "userId": 1, "role": "ROLE_USER", "iat": 1234567890, "exp": 1234654290 }
Signature: HMAC-SHA256(header + "." + payload, secret)
```

Secret Storage:

```
jwt:
  secret: ${JWT_SECRET:404E635266556A586E3272357538782F413F4428472B4B6250645367566B5970}
  expiration: ${JWT_EXPIRATION:86400000}
```

- Secret stored in **environment variable** (not hardcoded)
- Minimum 256-bit key for HS256
- Default value provided for development only

D. `JwtAuthenticationFilter.java` - Token Validation

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) {

        final String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        final String token = authHeader.substring(7);
        final String username = jwtService.extractUsername(token);

        if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails = userDetailsService.loadUserByUsername(username);

            if (jwtService.isTokenValid(token, userDetails)) {
                UsernamePasswordAuthenticationToken auth = new UsernamePasswordAuthenticationToken(
                    userDetails, null, userDetails.getAuthorities()
                );
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

Flow:

1. Extract JWT from `Authorization: Bearer {token}` header
2. Validate token signature and expiration
3. Load user from database
4. Set authentication in `SecurityContext` (Spring Security now knows the user)

E. `UserDetailsServiceImpl.java` - Spring Security Integration

```
@Service
public class UserDetailsServiceImpl implements UserDetailsService {

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));

        SimpleGrantedAuthority authority = new SimpleGrantedAuthority(user.getRole().getAuthority());

        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPasswordHash(),
            Collections.singletonList(authority)
        );
    }
}
```

Purpose: Bridges our `User` entity with Spring Security's `UserDetails` interface.

5. Authentication Endpoints

AuthController.java

Endpoint	Method	Description
/api/auth/register	POST	Register new user with validation
/api/auth/login	POST	Login and receive JWT
/api/auth/logout	POST	Logout (frontend handles)

AuthService.java - Core Logic

```

@Service
public class AuthService {

    public User register(RegisterRequest request) {
        // 1. Validate uniqueness (username, email, phone)
        validateUniqueness(request);

        // 2. Hash password with BCrypt
        String hashedPassword = passwordService.hashPassword(request.getPassword());

        // 3. Create user with ROLE_USER
        User user = new User(
            request.getUsername(),
            request.getEmail(),
            request.getPhone(),
            hashedPassword,
            Role.ROLE_USER
        );

        return userRepository.save(user);
    }

    public LoginResponse login(LoginRequest request) {
        // 1. Authenticate with Spring Security
        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(request.getUsername(), request.getPassword())
        );

        // 2. Generate JWT
        User user = userRepository.findByUsername(request.getUsername()).get();
        String token = jwtService.generateToken(user);

        return new LoginResponse(token, user.getId(), user.getUsername(), user.getEmail());
    }
}

```

6. Article Authorization

Updated ArticleService.java

```

public Article createArticle(CreateArticleRequest request, User author) {
    // Author is set to current authenticated user
    article.setAuthor(author);
    return articleRepository.save(article);
}

public Article updateArticle(Long id, CreateArticleRequest request, User currentUser) {
    Article article = getArticleById(id);

    // Check ownership
    if (!article.getAuthor().getId().equals(currentUser.getId())) {
        throw new ArticleAccessDeniedException("You don't own this article");
    }

    article.setTitle(request.getTitle());
    article.setAbstractText(request.getAbstractText());
    article.setBody(request.getBody());
    return articleRepository.save(article);
}

```

7. Exception Handling

New Custom Exceptions

Exception	HTTP Status	Use Case
<code>UsernameAlreadyExistsException</code>	409 CONFLICT	Duplicate username on registration
<code>EmailAlreadyExistsException</code>	409 CONFLICT	Duplicate email on registration
<code>PhoneAlreadyExistsException</code>	409 CONFLICT	Duplicate phone on registration
<code>InvalidCredentialsException</code>	401 UNAUTHORIZED	Wrong username/password
<code>ArticleAccessDeniedException</code>	403 FORBIDDEN	User doesn't own the article
<code>JwtAuthenticationException</code>	401 UNAUTHORIZED	Invalid or expired JWT

`GlobalExceptionHandler.java`

```
@ControllerAdvice
public class GlobalExceptionHandler {
    // Each exception mapped to appropriate HTTP status
}
```

8. DTOs for Auth

Request DTOs

- `RegisterRequest` - username, password, email, phone
- `LoginRequest` - username, password
- `UpdateProfileRequest` - email, phone
- `ChangePasswordRequest` - currentPassword, newPassword

Response DTOs

- `LoginResponse` - accessToken, tokenType, userId, username, email
- `ProfileResponse` - id, username, email, phone, articles list

9. Data Initialization

Updated `DataInitializer.java`

```
@Component
@Profile("!test")
public class DataInitializer implements CommandLineRunner {

    @Override
    public void run(String... args) {
        // Only runs if no users exist
        if (userRepository.count() == 0) {
            // Create users with hashed passwords
            User user1 = new User(
                "alirezav",
                "alireza@example.com",
                "+989123456789",
                passwordEncoder.encode("SecurePass123"),
                Role.ROLE_USER
            );
            userRepository.save(user1);

            // ... other users and articles
        }
    }
}
```

10. Security Flow Diagram

AUTHENTICATION FLOW

REGISTRATION:

POST /api/auth/register

↓

Validate uniqueness (username, email, phone)

↓

Hash password with BCrypt (salt auto-generated)

↓

Create User with ROLE_USER

↓

Return 201 Created

LOGIN:

POST /api/auth/login

↓

Spring Security AuthenticationManager

↓

UserDetailsService loads user from DB

↓

PasswordEncoder verifies (BCrypt compares with hash)

↓

JWT generated with claims (userId, username, role)

↓

Return 200 OK with JWT token

AUTHENTICATED REQUEST:

Request with "Authorization: Bearer {token}"

↓

JwtAuthenticationFilter

↓

Extract & validate JWT

↓

Set Authentication in SecurityContext

↓

Request reaches Controller

↓

Controller uses SecurityContext to get user

↓

Authorization checks (article ownership)

↓

Return response

Summary of Security Features

Feature	Implementation
Password Storage	BCrypt with 12 work factor, salt auto-generated
Authentication	JWT with HS256, 24-hour expiration
Authorization	Role-based (ROLE_USER, ROLE_ADMIN) + Ownership checks
Session Management	STATELESS (no sessions)
Security Headers	Default Spring Security headers
CORS	Configured for frontend origins (5173, 3000)

Password Validation Feature	Min 6 chars, uppercase, lowercase, digit
Error Handling	Consistent error responses with codes
Testing	Full test coverage for security flows

Frontend Authentication & Authorization - Technical Changes Summary

Overview

We implemented a **complete authentication flow** on the frontend to complement the backend JWT-based authentication system. The frontend handles user registration, login, session management, protected routes, and conditional UI rendering based on authentication state.

1. Authentication Context & State Management

AuthContext.jsx - Centralized Auth State

```
// Context provider that wraps the entire app
const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(null);
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [loading, setLoading] = useState(true);

  // Initialize auth state from localStorage on mount
  useEffect(() => {
    const storedToken = localStorage.getItem('authToken');
    const storedUser = localStorage.getItem('authUser');
    // Verify token validity with backend
  }, []);

  // Auth methods
  const login = useCallback(async (credentials) => { ... });
  const register = useCallback(async (userData) => { ... });
  const logout = useCallback(() => { ... });
  const updateProfile = useCallback(async (profileData) => { ... });
  const changePassword = useCallback(async (passwordData) => { ... });
};
```

Key Design Decisions:

- **Single source of truth** – All auth state in one context
- **Token persistence** – JWT stored in localStorage (with security awareness)
- **Auto-verification** – Token validity checked on app initialization
- **Loading state** – Prevents flash of unauthenticated content

2. Secure Token Storage Strategy

Token Storage Implementation

```
// Token stored in localStorage with security considerations
localStorage.setItem('authToken', response.accessToken);
localStorage.setItem('authUser', JSON.stringify(response));
```

Security Awareness:

Aspect	Implementation

Storage Aspect	localStorage (simpler, XSS vulnerable) Implementation
Token removal	Cleared on logout and 401 responses
Session management	Frontend handles token lifecycle
Production consideration	httpOnly cookies recommended for production

3. Axios Interceptors & API Integration

axiosConfig.js - Request/Response Interceptors

```
// Request interceptor - adds token to all requests
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('authToken');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Response interceptor - handles 401 errors
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (response?.status === 401) {
      // Clear token and redirect to login
      localStorage.removeItem('authToken');
      localStorage.removeItem('authUser');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

4. Authentication Pages

Login Page

```
// Features
- Username + password form
- Real-time validation
- Error handling with toast notifications
- Redirect to previous page after login
- Loading states
```

Register Page

```
// Features
- Username, email, phone, password fields
- Real-time validation with feedback
- Password strength indicator (optional)
- Email and phone format validation
- Redirect to login on success
```

Validation Rules Applied:

Field	Rules
Username	Required, min 3, max 50, alphanumeric + underscore
Email	Required, valid format
Phone	Optional, valid format (e.g., +989123456789)
Password	Min 6, uppercase, lowercase, digit

5. Profile Page

ProfilePage.jsx - User Management

```
// Features
- Display user info: username, email, phone
- List of user's published articles with links
- Edit profile: inline editing of email and phone
- Change password: form with current + new password
- Logout functionality
```

6. Protected Route Component

ProtectedRoute.jsx - Route Protection

```
const ProtectedRoute = ({ children, requireAuth = true }) => {
  const { isAuthenticated, loading } = useAuth();
  const location = useLocation();

  if (loading) return <div>Loading...</div>;

  // Protected route - requires authentication
  if (requireAuth && !isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  // Public route - redirect to home if already authenticated
  if (!requireAuth && isAuthenticated) {
    return <Navigate to="/" replace />;
  }

  return children;
};
```

Protected Routes in App:

Route	Protection
/add	Requires authentication
/profile	Requires authentication
/login	Redirects to home if authenticated
/register	Redirects to home if authenticated

/ (home) Route	Public Protection
/article/:id	Public
/popular	Public

7. Conditional UI Rendering

Authentication-Aware Components

Navbar.jsx:

```
{isAuthenticated ? (  
  <UserMenu user={user} onLogout={handleLogout} />  
) : (  
  <div className={styles.authButtons}>  
    <Link to="/login">Log In</Link>  
    <Link to="/register">Sign Up</Link>  
  </div>  
)}
```

HomePage.jsx:

```
{isAuthenticated && (  
  <Link to="/add" className={styles.addButton}>  
    ✎ Write New Article  
  </Link>  
)}
```

ArticleCard.jsx:

```
const canEdit = isAuthenticated && isArticleAuthor(article, user);  
  
{canEdit && (  
  <div className={styles.actionButtons}>  
    <button onClick={handleEdit}>✎ Edit</button>  
    <button onClick={handleDelete}>🗑 Delete</button>  
  </div>  
)}
```

Conditional UI Logic:

Condition	Shows
User not authenticated	Login/Register links
User authenticated	Profile/Logout links + "Write" button
User is article author	Edit/Delete buttons
User is not author	No Edit/Delete buttons

8. Form Validation

validators.js - Centralized Validation

```
// Reusable validation functions
export const validateUsername = (value) => { ... };
export const validateEmail = (value) => { ... };
export const validatePhone = (value) => { ... };
export const validatePassword = (value) => { ... };
export const validateConfirmPassword = (password, confirm) => { ... };

// Form-specific validation
export const validateLogin = (data) => { ... };
export const validateRegistration = (data) => { ... };
export const validateProfile = (data) => { ... };
export const validatePasswordChange = (data) => { ... };
```

Validation Flow:

1. **On blur** – Field-level validation with immediate feedback
2. **On submit** – Full form validation before submission
3. **Real-time** – Password strength indicator updates as user types

9. API Services

authApi.js - Authentication API Calls

```
export const register = async (userData) => {
  const response = await api.post('/auth/register', userData);
  return response.data;
};

export const login = async (credentials) => {
  const response = await api.post('/auth/login', credentials);
  return response.data;
};

export const getCurrentUser = async () => {
  const response = await api.get('/users/profile');
  return response.data;
};

export const updateProfile = async (profileData) => {
  const response = await api.put('/users/profile', profileData);
  return response.data;
};

export const changePassword = async (passwordData) => {
  const response = await api.put('/users/password', passwordData);
  return response.data;
};
```

10. Toast Notifications & Error Handling

User Feedback System

```
// Success messages
toast.success('📄 Article created successfully!');
toast.success('Password changed successfully!');
toast.success('Welcome back!');

// Error messages
toast.error('Please login to create an article');
toast.error("You don't have permission to edit this article");
toast.error('Invalid username or password');
```

Toast Messages by Scenario:

Scenario	Message
Login success	"Welcome back!"
Login failure	"Invalid username or password"
Registration success	"Account created successfully! Please login."
Profile update	"Profile updated successfully!"
Password change	"Password changed successfully!"
Unauthorized action	"Please login to create an article"
Permission denied	"You don't have permission to edit this article"

11. Frontend Security Flow Diagram

FRONTEND AUTHENTICATION FLOW

REGISTRATION:

User fills Register form
↓
Validation (username, email, phone, password)
↓
POST /api/auth/register
↓
Success → toast + redirect to login
↓
Error → display validation/API error

LOGIN:

User fills Login form
↓
Validation (username, password)
↓
POST /api/auth/login
↓
Success → store token in localStorage
↓
Set user in AuthContext
↓
Redirect to homepage
↓
Error → display error toast

AUTHENTICATED REQUESTS:

Any API request
↓
Axios interceptor adds Authorization header
↓
Request sent with Bearer token
↓
401 response → clear token + redirect to login

PROTECTED ROUTES:

User navigates to /add or /profile
↓
ProtectedRoute checks isAuthenticated
↓
Not authenticated → redirect to /login
↓
Authenticated → render page

LOGOUT:

User clicks logout button
↓
Clear token from localStorage
↓
Clear user from AuthContext
↓
Redirect to homepage

12. Folder Structure for Auth


```
frontend/src/
├── context/
│   └── AuthContext.jsx           # Auth state & provider
├── components/
│   └── ProtectedRoute.jsx       # Route protection wrapper
├── pages/
│   ├── LoginPage.jsx           # Login form
│   ├── RegisterPage.jsx        # Registration form
│   └── ProfilePage.jsx          # User profile & settings
├── services/
│   ├── axiosConfig.js          # Axios interceptors
│   └── authApi.js              # Auth API calls
├── utils/
│   ├── validators.js           # Form validation
│   └── helpers.js              # Helper utilities
└── App.jsx                     # Routes with protection
```

Summary of Frontend Security Features

Feature	Implementation
Authentication State	React Context API (AuthContext)
Token Storage	localStorage (with security awareness)
API Interceptors	Axios for automatic token injection
Protected Routes	ProtectedRoute wrapper component
Conditional UI	Auth-aware rendering in components
Form Validation	Centralized validators with real-time feedback
Error Handling	Toast notifications for user feedback
Password Strength	Visual indicator during registration
Session Management	Automatic logout on 401 responses
Testing	Comprehensive unit and integration tests

Your frontend is now **fully integrated** with the backend authentication system! 🎉